electronics

MDPI

Article

Enhancing Neural Collaborative Filtering for Product Recommendation by Integrating Sales Data and User Satisfaction

Haoyang Xia [1,2,*] and Yuanyuan Wang [1,*]

[1] Graduate School of Sciences and Technology for Innovation, Yamaguchi University, Ube 755-8611, Japan
[2] MICWARE Co., Ltd., Kobe 650-0035, Japan
* Correspondence: f003wcw@yamaguchi-u.ac.jp (H.X.); y.wang@yamaguchi-u.ac.jp (Y.W.)

Abstract

The rapid growth of e-commerce has made it increasingly difficult for users to select appropriate products due to the overwhelming amount of available information. Although many platforms, such as Amazon and Rakuten, encourage users to leave reviews, effectively utilizing this information for personalized recommendations remains a challenge. To address this issue, we propose a multi-task product recommender system that supports both new users without purchase histories and existing users with interaction records. For new users without purchase histories, we introduce a ranking-based method that combines three market-oriented features: sales volume, sales period, and user satisfaction. User satisfaction is quantified using sentiment analysis of product reviews. These three factors are integrated into a composite score to identify products with a strong market presence and positive customer feedback. For existing users, we developed an enhanced neural collaborative filtering (NCF) method that incorporates a product bias factor. This model, named bias neural collaborative filtering (BNCF), utilizes multilayer perceptrons to learn latent user–product interactions while also capturing item popularity bias. We evaluated the proposed approaches using a real-world dataset from Rakuten. The results show that our multi-task system effectively improves recommendation quality for users in both cold-start and data-rich scenarios.

Keywords: neural collaborative filtering; bias-aware recommendation; cold-start user recommendation; personalized product ranking; sentiment-aware recommendation; Bayesian personalized ranking; deep learning for recommender systems

Academic Editor: George Angelos Papadopoulos

Received: 8 July 2025
Revised: 3 August 2025
Accepted: 5 August 2025
Published: 8 August 2025

Citation: Xia, H.; Wang, Y. Enhancing Neural Collaborative Filtering for Product Recommendation by Integrating Sales Data and User Satisfaction. *Electronics* **2025**, *14*, 3165. https://doi.org/10.3390/electronics14163165

1. Introduction

The rapid growth of e-commerce has significantly altered consumer behavior by providing the convenience of a wide selection of products and a personalized shopping experience. A key technology supporting this transformation is the recommender system, which helps users to discover relevant products by analyzing their preferences, behavior, and contextual data. Product reviews and ratings are valuable sources of user information because they contribute to product credibility by providing subjective feedback. Lackermair et al. [1] demonstrated that online reviews and ratings directly impact purchase intentions, establishing them as a primary source of consumer decision-making. However, due to the volume and unstructured nature of this feedback, users often struggle to process it efficiently. Changchit et al. [2] further emphasized that integrating customer reviews with product specifications enhances customer satisfaction and influences purchasing decisions.

Although review data is valuable, it is insufficient to solve all recommendation challenges, especially those involving new users with limited historical interactions. To address this cold-start problem, we propose a recommendation approach that integrates three market-oriented product attributes: sales volume, sales period, and user satisfaction. The user satisfaction score is calculated using sentiment analysis of product reviews and rating distributions. These attributes are combined using a weighted ranking model with threshold-based filtering. It enables the system to recommend products that are both popular and have positive customer reviews.

Many recommender systems, particularly those on e-commerce platforms, are based on collaborative filtering (CF). However, the effectiveness of CF is often influenced by user search and browsing behavior. Illm et al. [3] pointed out that CF performance degrades when users explore broad product categories or lack specific interests. To address data sparsity and improve accuracy, researchers have extensively studied hybrid approaches that combine CF with content-based filtering. Çano et al. [4] demonstrated that these combinations effectively mitigate the cold-start problem and enhance personalization. Additionally, Sarwar et al. [5] showed that item-based CF methods outperform user-based methods in terms of scalability and robustness.

Recent advancements in deep learning have further expanded the capabilities of recommender systems. Deep neural networks (DNNs), which are commonly used for image recognition and natural language processing, can now model user–item interactions. These models can capture complex nonlinear relationships and learn from implicit feedback data. According to Ko et al. [6], the use of deep learning has improved recommender performance by addressing issues such as data sparsity and hidden user preferences. However, challenges remain, particularly regarding model scalability and interpretability. As consumer behavior becomes more dynamic and context-dependent, recommendation algorithms must incorporate both implicit behavioral signals and explicit market indicators.

In this study, we propose a dual-framework recommender system that addresses both cold-start and data-rich scenarios. For new users without purchase histories, we propose a ranking-based approach for recommending products that considers product attributes, such as sales volume, sales period, and user satisfaction. For existing users with historical interactions, we develop a bias neural collaborative filtering (BNCF) model that incorporates item-level popularity factors into the recommendation process. The detailed workflow and model architecture are described in Section 3.

In summary, our contributions are twofold:

- We propose a ranking-based recommendation approach for new users that integrates sentiment analysis with sales-oriented features. This approach helps to overcome the cold-start problem without relying on user history.
- We introduce a bias-aware neural collaborative filtering model that incorporates product-level popularity factors to improve recommendation performance for users with prior interactions.

The remainder of this paper is structured as follows: Section 2 reviews the related work on traditional and deep learning–based recommender systems, emphasizing sentiment analysis and hybrid models. Section 3 describes the proposed recommendation approaches for both new and existing users, including feature extraction and model design. Section 4 presents the experimental results, including performance evaluations, ablation studies, and hyperparameter analyses. Section 5 discusses the findings and compares the proposed methods. Finally, Section 6 concludes the paper and outlines directions for future research.

2. Related Work

To position our contribution within the broader research landscape, we review the existing studies across three key themes: (1) cold-start and hybrid recommendation strategies, (2) neural collaborative filtering and bias-aware models, and (3) sentiment-aware and multimodal approaches in recommender systems.

2.1. Cold-Start and Hybrid Recommendation Methods

The cold-start recommendation problem has been addressed through content-based filtering, popularity metrics, hybrid methods, and rule-based logic. Traditional collaborative filtering struggles when there is insufficient user history. To solve this issue, researchers have proposed hybrid models that incorporate demographic features, purchase history, and temporal behavior [7,8].

Recent methods integrate additional side information, such as user behavior, contextual cues, and external metadata. For example, Adeniyi et al. [8] employed clickstream data and K-nearest neighbor (KNN) to develop real-time content-based recommender systems. Zou et al. [9] employed formal concept analysis to overcome sparsity. Wu et al. [10] integrated temporal and contextual factors via Bayesian modeling and hierarchical clustering. These approaches demonstrate that combining content, behavior, and structure improves cold-start performance.

Our approach builds on this hybrid logic by combining product-level popularity metrics, such as sales volume and period, with sentiment-derived satisfaction to create a multidimensional ranking score. This method effectively recommends products to new users without requiring interaction data.

2.2. Neural Collaborative Filtering and Bias-Aware Enhancements

Deep learning has revolutionized recommender systems by capturing high-order interactions and nonlinear dependencies. He et al. [11] introduced neural collaborative filtering (NCF), in which the inner product of matrix factorization is replaced with multilayer perceptrons (MLPs). This allows for richer feature representations. Extensions of NCF, such as DeepNCF [12] and Wide & Deep [13], combine generalization and memorization.

Other works enhance traditional collaborative filtering using convolutional neural networks (CNNs) [14] or recurrent architectures (e.g., LSTM) to model temporal dynamics [15]. Bobadilla et al. [7] emphasized the importance of bias modeling in BiasedMF [16], which explicitly incorporates user/item rating deviations.

While these models improve personalization, few directly integrate product-level popularity or bias as structured input. Our BNCF model extends NCF by incorporating item-side attributes (sales volume, period, and satisfaction) as bias signals into the embedding and MLP layers, improving interpretability and cold-start robustness.

2.3. Sentiment, NLP, and Multimodal Integration in Recommendations

Natural language processing (NLP) has become a popular method for extracting insights from user reviews. Redhu et al. [17] and Pham et al. [18] demonstrated that sentiment analysis enhances the performance of recommender systems by capturing emotional and experiential feedback that is not evident in ratings alone. Models such as deep cooperative neural networks (DeepCoNNs) [19] and the review and content-based deep fusion model (RC-DFM) [20] use convolutional neural networks (CNNs) or autoencoders to fuse reviews with product metadata.

Transformer-based models (e.g., BERT) have recently improved sentiment understanding and semantic modeling in recommendation tasks [21,22]. Xu et al. [21] combined BERT with KNN to enhance semantic similarity, while Bellar et al. [22] compared BERT with FastText for emotion-rich predictions.

Our method builds on this foundation by using the Google Cloud Natural Language API for sentiment scoring, which we integrate with user ratings to compute a satisfaction score. This score serves as one dimension of our composite ranking model for cold-start users. Meanwhile, multimodal models such as BoNMF [23] and RC-DFM [20] demonstrate the power of integrating text, image, and behavioral data. Although these systems are impressive, they often require substantial computing power. Our approach integrates rich signals through simpler scalar metrics derived from sentiment, ratings, and market data, maintaining interpretability and efficiency.

3. Dual-Approach Recommender System Design

This section outlines our recommendation strategies for two types of users: new users without purchase histories and returning users with purchase records. Different methods are necessary to address the diversity of user behavior on e-commerce platforms and to improve the accuracy and relevance of product recommendations.

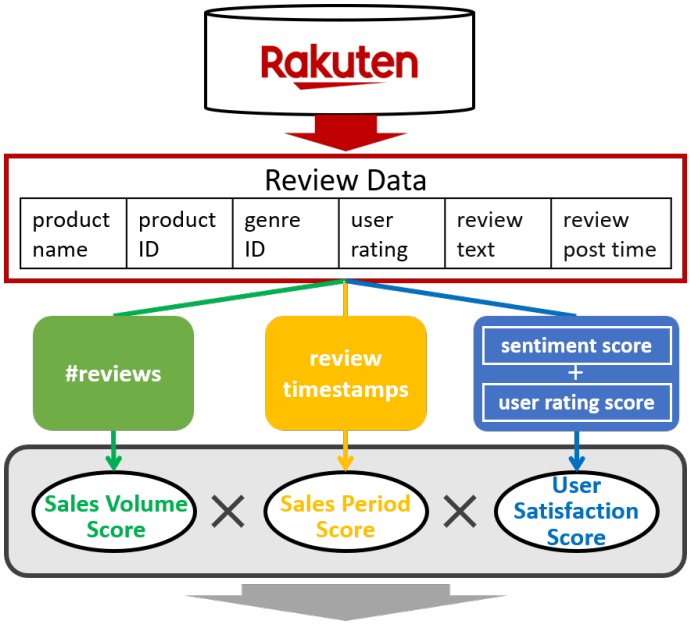
Poriya et al. [24] distinguish between non-personalized and user-based collaborative filtering recommender systems. Non-personalized systems use aggregate metrics, such as product popularity or average ratings, to generate recommendations that are the same for all users. In contrast, user-based collaborative filtering uses algorithms, such as the Pearson correlation coefficient, to analyze similarities between users and recommend products based on the preferences of similar users.

In this study, we propose a dual recommendation strategy. For new users without purchase histories, we utilize a content- and behavior-based approach that integrates product popularity and user feedback. For users with existing purchase data, we apply a neural collaborative filtering model that incorporates user and product bias factors. Our goal is to use natural language processing and deep learning techniques to provide personalized product recommendations to both user types, thereby enhancing the overall shopping experience.

The following subsections describe our approach in detail. Section 3.1 presents the recommendation method for new users. It explains the data selection and preprocessing steps, defines the evaluation metrics of product performance, and introduces the composite recommendation formula. Section 3.2 covers the method for returning users, including data acquisition, constructing the BNCF model, and extracting user- and product-specific features.

3.1. Product Recommendation Method for New Users Without Purchase Histories

This section focuses on recommendation methods for new users who have recently registered on the platform but have not yet made any purchases. In such cases, the absence of purchase behavior and user-specific attributes makes it hard to infer preferences through conventional personalized approaches. Therefore, we propose a method that generates recommendations using objective product performance indicators, such as sales data, user reviews, and market longevity. Figure 1 illustrates the overall workflow of our product ranking system for cold-start users.



Rank	Product Name
1	20-Inch Hardside Spinner
2	25-Inch Softside Expandable Roller Luggage
3	Softside Upright Luggage Set
4	28-Inch Hardside Spinner
5	Luggage Suitcases With Spinner Wheels
⋮	⋮

Figure 1. Workflow of the proposed product ranking method for new users without purchase histories.

In general, products with high sales figures tend to have widespread market acceptance, reflecting qualities such as high quality, good value, and satisfactory performance. Similarly, products that maintain high sales over extended periods demonstrate sustained customer satisfaction and reliability. To capture these aspects, we introduce a recommendation strategy that evaluates products based on three dimensions: sales volume, sales period, and user satisfaction.

We constructed a multidimensional hybrid recommendation model that integrates content-based sentiment analysis with sales and temporal data. This model represents each product in a three-dimensional space, where the axes correspond to the normalized values of sales volume, sales period, and user satisfaction. Products are plotted in this space according to their performance across these attributes. A product that scores highly on all three dimensions occupies a position in the ideal region of this space, visualized in Figure 2 as a red cube. Our recommendation algorithm prioritizes these products, aiming to present users with options that are both popular and well-reviewed over time.

The methodology begins with the collection and preparation of product metadata. When direct sales data is unavailable, we define proxy metrics for sales volume and sales period using the number and timing of user reviews. We also apply sentiment analysis to review texts to quantify user satisfaction. These metrics are then integrated into a unified recommendation score that ranks products according to their overall performance across all three dimensions.

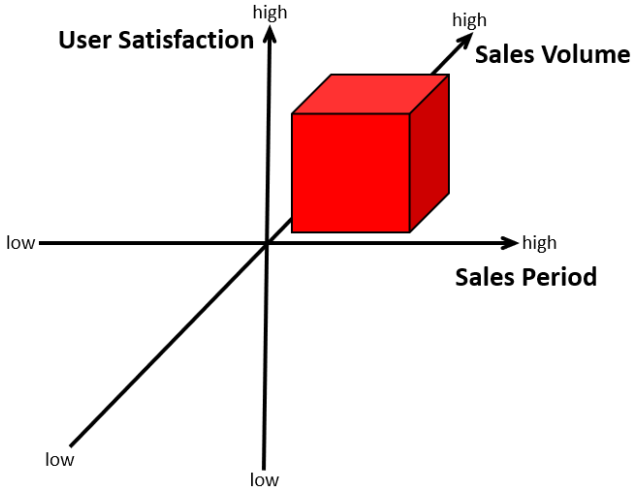


Figure 2. Product recommendation based on sales volume, sales period, and user satisfaction.

3.1.1. Sales Volume Scoring

Due to the unavailability of direct sales data on the Rakuten website, we used the number of user reviews as a proxy for sales volume. Previous studies have demonstrated a strong correlation between the number of reviews and the number of purchases [25,26]. Based on this relationship, we calculated a sales volume score ($Vol(a)$) for a target product a using the total number of reviews.

However, we acknowledge the limitations of using review count as a practical proxy for sales volume. Depending on factors such as product type, price, longevity on the platform, and customer engagement patterns, users may be more or less likely to leave reviews. It can introduce bias into sales estimates. For example, products that elicit strong positive or negative opinions, or that are part of incentivized review campaigns, may receive a disproportionate number of reviews. Although we acknowledge the possibility of bias, using review count is commonly accepted and necessary when direct sales data is unavailable. Future work could evaluate methods that incorporate review conversion rates or supplement this proxy with additional behavioral signals to create a more accurate representation of sales velocity.

To classify products, we used the average number of reviews across all products as a threshold value. We chose this data-driven approach because it provides a simple, replicable, and objective baseline for distinguishing between popular and less popular items in our dataset. It is especially crucial since there are no established industry benchmarks for this product category. Products that met or exceeded this threshold value were labeled as having "high sales volume," while those that fell below were labeled as "low sales volume." Using this criterion, all products are appropriately classified based on their sales volume. While a formal sensitivity analysis to test different threshold values was not conducted as part of this study, we acknowledge that exploring the impact of this threshold on model performance is a valuable direction for future work.

3.1.2. Sales Period Scoring

Similar to sales volume, the Rakuten website did not provide direct data on product availability periods. Therefore, we estimated the sales period of each product by calculating the time interval between its first and last review. We used this interval as the estimated sales duration of each product.

We then assigned a sales period score ($Per(a)$) to the target product a. Products were classified as having either a "long" or "short" sales period, using the average sales period across all products as the benchmark.

3.1.3. User Satisfaction Scoring

We assessed user satisfaction by combining review sentiment and user ratings. We performed a sentiment analysis on the collected reviews using the Google Cloud Natural Language API (https://cloud.google.com/natural-language) (accessed on 22 June 2023), which assigns sentiment scores ranging from −1.0 (most negative) to 1.0 (most positive). To ensure reproducibility, Table 1 lists the key configuration settings used for the sentiment analysis. Unless otherwise specified, we used the default settings provided by the Google Cloud Natural Language API. The study was conducted in English in document mode using the analyzeSentiment method.

Table 1. Google Cloud Natural Language API configuration for sentiment analysis.

Parameter	Value / Setting
API Method	analyzeSentiment
Document Type	PLAIN_TEXT
Mode	Sentence-level
Language	ja (Japanese)
Encoding Type	UTF-8
Sentiment Score Range	−1.0 (most negative) to +1.0 (most positive)
Sentiment Magnitude	Included but not used in scoring
Threshold for Positive Sentiment	>0.25
Threshold for Neutral Sentiment	$-0.25 \leq$ score ≤ 0.25
Threshold for Negative Sentiment	<−0.25

Based on standard practices in sentiment analysis and the API's documentation, we categorized reviews into three groups: negative (−1.0 to −0.25), neutral (−0.25 to 0.25), and positive (0.25 to 1.0). These thresholds are widely accepted for partitioning sentiment scores into distinct emotional classes. Each review was classified accordingly, and the sentiment score was combined with the user rating to derive a satisfaction score.

We tested four methods of calculating a satisfaction score for each product by combining sentiment scores and user ratings, as determined by a preliminary experiment [27]. The most effective method is to multiply the sentiment score by the user rating for each review and then aggregate these values. The final satisfaction score for the target product a is calculated using sentiment scores and user ratings, as defined in Equation (1):

$$Sat(a) = \frac{\sum_{i=1}^{n} P_i R_i}{\sum_{i=1}^{n} P_i R_i + \sum_{i=1}^{n} |N_i| R_i + \sum_{i=1}^{n} |Z_i| R_i} \quad (1)$$

Here, P_i, N_i, and Z_i represent the positive, negative, and neutral sentiment scores, respectively, for the ith review (with absolute values for N_i and Z_i). R_i is the corresponding user rating. The sum is computed over all reviews n for product a. Similar to our sales volume scoring, we used the average satisfaction score across all products as a data-driven threshold to categorize products. Products with satisfaction scores above the average were classified as "high satisfaction" and others as "low satisfaction." Using this criterion, all target products were categorized accordingly. Future work could include a sensitivity analysis to evaluate how different satisfaction thresholds might influence the final recommendation rankings.

3.1.4. Product Recommendation Ranking

Traditional recommendation models often rely solely on individual attributes, such as product category, brand, or user ratings. While these methods are beneficial, they may overlook key behavioral and temporal signals. To improve relevance and fairness, we integrated sales volume, sales period, and satisfaction into a unified ranking model.

Since these attributes have different scales, where satisfaction ranges from 0 to 1, sales volume spans a wide numerical range, and sales periods extend from 0 to 365 days, we then normalized each dimension to ensure equal influence. The final recommendation score ($Score(a)$) for the target product a is calculated as shown in Equation (2):

$$Score(a) = Vol(a) \times Per(a) \times Sat(a) \tag{2}$$

Our choice regarding a multiplicative aggregation for the composite score is deliberate. We hypothesize that the top recommended product for a new user should excel in all three dimensions: popularity (sales volume), longevity (sales period), and quality (user satisfaction). A multiplicative approach ensures that products with low scores in any of these areas will be appropriately penalized in the final ranking. For example, a product with high sales volume but poor user satisfaction should not receive a high recommendation. Multiplication naturally captures this 'AND' relationship between attributes. In contrast, an additive model allows a high score in one dimension to compensate for a low score in another.

This composite score reflects a product's market performance, customer satisfaction, and longevity. Products are ranked based on this score to provide new users with balanced recommendations that consider both objective metrics and subjective feedback.

While this multiplicative model provides a robust and interpretable baseline, we acknowledge that its heuristic nature is a limitation. We could explore alternative aggregation methods. For example, a weighted linear combination, $Score(a) = w_1 \cdot Vol(a) + w_3 \cdot Per(a) + w_3 \cdot Sat(a)$, would enable more precise control over the importance of each attribute. Furthermore, incorporating advanced learnable or adaptive weighting mechanisms, such as those based on linear regression or neural networks, could improve performance because the optimal weighting scheme may depend on the user or the context. Exploring such adaptive approaches is a promising direction for future research.

3.2. Product Recommendation Method for Existing Users with Purchase Histories

Collaborative filtering is a widely used strategy for users with an established purchase history. It uses historical data on user–product interactions to improve recommendation accuracy. To further enhance recommendation performance, we propose a bias neural collaborative filtering (BNCF) model. The BNCF model is based on the traditional neural network-based collaborative filtering (NCF) framework and incorporates three product-related bias factors: sales volume, sales period, and user satisfaction. The architecture of the BNCF model is shown in Figure 3.

First, we calculate the bias scores for each product using the methodology described in Section 3.1. Then, we incorporate these scores into the BNCF model alongside the neural feature extraction layers. The BNCF model consists of embedding, hidden, and output layers. These layers are designed to capture latent user–product interactions. Finally, we calculate the recommendation score by combining interaction features and bias-based scores. We train the model using a Bayesian personalized ranking (BPR) loss function, which models user preferences through pairwise ranking and enables personalized optimization. The remainder of this subsection provides a detailed explanation of the BNCF model design and training process.

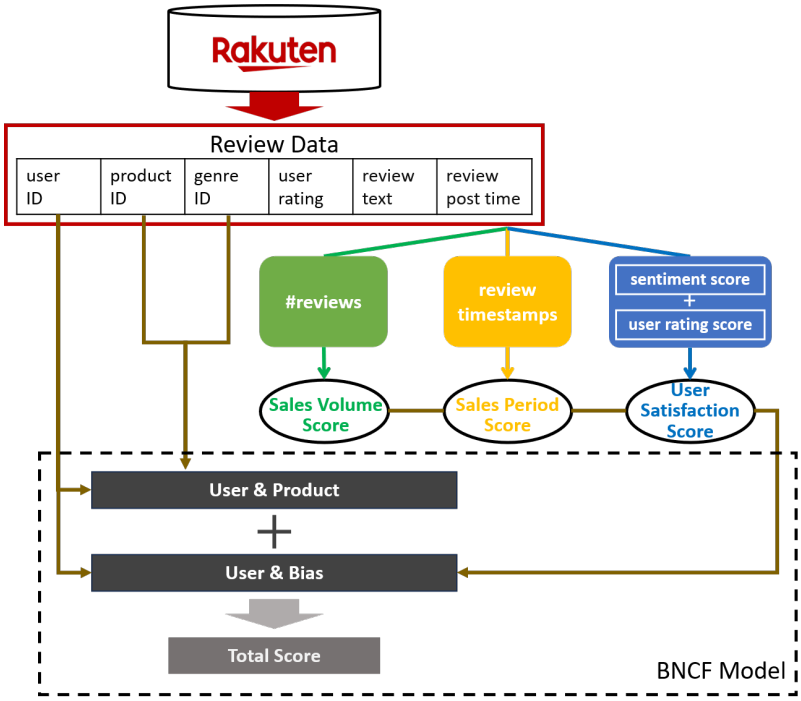


Figure 3. Overview of the proposed BNCF model for existing users with purchase histories.

3.2.1. Construction of BNCF Model

The neural collaborative filtering (NCF) model employs embedding layers to map users and products onto a shared low-dimensional latent space. It also uses a multilayer perceptron (MLP) to model complex nonlinear interactions. Our BNCF model builds on this by incorporating product and user bias information into the NCF framework, thereby improving the accuracy of personalized recommendations. The architecture is depicted in Figure 4.

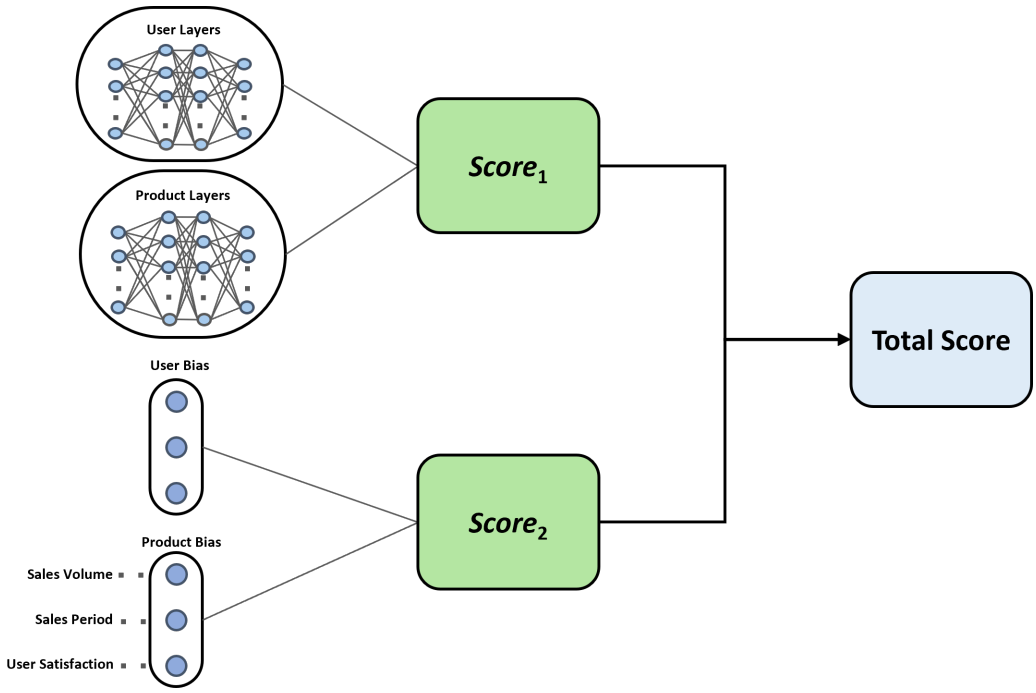


Figure 4. BNCF model architecture. The model computes two scores in parallel. $Score_1$ is the collaborative filtering prediction from the NCF module, which processes learned user and item embeddings (represented by the interconnected nodes). $Score_2$ is the bias-aware prediction, derived from the explicit bias feature embeddings (represented by the blue circles).

The BNCF model consists of two main components:

1. **Extraction of User and Product Latent Features**
First, we initialize the user and product embedding vectors. Each vector is n-dimensional. Then, we pass these embeddings through two fully connected layers to increase feature expressiveness. Specifically,

$$e_u = \{e_{u_1}, e_{u_2}, e_{u_3}, \ldots, e_{u_n}\}, e_i = \{e_{i_1}, e_{i_2}, e_{i_3}, \ldots, e_{i_n}\}$$

These vectors are processed through two-layer neural networks with ReLU activation to produce refined feature vectors, h_u and h_i, which are defined as shown in Equations (3) and (4), respectively:

$$h_u^{(2)} = \text{ReLU}\left(W_u^{(2)} \cdot \text{ReLU}\left(W_u^{(1)} e_u + b_u^{(1)}\right) + b_u^{(2)}\right) \quad (3)$$

$$h_i^{(2)} = \text{ReLU}\left(W_i^{(2)} \cdot \text{ReLU}\left(W_i^{(1)} e_i + b_i^{(1)}\right) + b_i^{(2)}\right) \quad (4)$$

- In our study, $h_u^{(2)}$ and $h_i^{(2)}$ represent the final hidden layer outputs for user u and item i, respectively.
- ReLU (rectified linear unit) is a commonly used activation function, defined as $\text{ReLU}(x) = \max(0, x)$.
- e_u and e_i are the embedding vectors for user u and item i, respectively.
- $W_u^{(1)}$ and $W_u^{(2)}$ denote the weight matrices for the first and second layers of the user network.
- $b_u^{(1)}$ and $b_u^{(2)}$ represent the bias terms for the first and second layers of the user network.
- $W_i^{(1)}$ and $W_i^{(2)}$ denote the weight matrices for the first and second layers of the item network.
- $b_i^{(1)}$ and $b_i^{(2)}$ represent the bias terms for the first and second layers of the item network.

After obtaining the feature vectors h_u and h_i for the user and the item, respectively, the first interaction score ($Score_1$) is obtained by computing the dot product of these two vectors as shown in Equation (5).

$$Score_1 = h_u^{(2)} \cdot h_i^{(2)} = \sum_{k=1}^{n} h_{u_k} h_{i_k} \quad (5)$$

2. **Incorporation of User and Product Bias Features**
Each user is assigned a three-dimensional bias embedding vector:

$$a_u = \{a_{u_1}, a_{u_2}, a_{u_3}\}$$

Each product is similarly represented using three normalized bias values corresponding to sales volume, sales period, and user satisfaction:

$$a_i = \{a_{i_1}, a_{i_2}, a_{i_3}\}$$

Next, the second interaction score ($Score_2$) is computed by taking the dot product of the user bias vector (a_u) and the product bias vector (a_i). It reflects the bias in the user–product relationship. It improves the accuracy and flexibility of the model as an additional component of the final recommendation score in Equation (6).

$$Score_2 = a_u \cdot a_i = \sum_{k=1}^{3} a_{u_k} a_{i_k} \tag{6}$$

The final prediction score is the sum of the two components mentioned above, as shown in Equation (7).

$$Total\ Score = Score_1 + Score_2 = \sum_{k=1}^{n} h_{u_k} h_{i_k} + \sum_{k=1}^{3} a_{u_k} a_{i_k} \tag{7}$$

This formulation allows the model to generate more accurate context-aware recommendations by using learned latent features and explicit bias signals.

3.2.2. Bayesian Personalized Ranking Loss

To optimize the BNCF model, we used the Bayesian personalized ranking (BPR) loss function [28], which is suitable for tasks involving implicit feedback recommendations. BPR formulates preference learning as a ranking problem, ensuring that observed (positive) items are ranked higher than unobserved (negative) items for a given user.

The objective of the training is to maximize the difference in preference between positive and negative item pairs using a sigmoid-based loss function. For a given user u, a positive item i, and a negative item j, the BPR loss is defined as in Equation (8):

$$\mathcal{L}_{\text{BPR}} = -\sum_{(u,i,j)\in D} \log \sigma(\hat{x}_{u,i} - \hat{x}_{u,j}) \tag{8}$$

This formulation enables the model to optimize personalized rankings by updating latent factors that directly reflect user preferences.

3.2.3. Model Training

We set the embedding dimensionality to $n = 32$ and incorporated normalized sales volume, sales period, and user satisfaction as bias inputs. During training, we generated user–item triplets (user u, positive item i, and negative item j) and trained the model to satisfy the inequality $\hat{x}_{u,i} > \hat{x}_{u,j}$.

The model was initialized with random parameters and optimized using the Adam optimizer [29]. The hyperparameters were configured as follows:

- Learning rate: 0.001;
- Weight decay: 0.00001;
- Mini-batch size: 256;
- Epochs: 200.

The model parameters were updated using mini-batch stochastic gradient descent and backpropagation until convergence was reached. Figure 5 illustrates the training flow.

To evaluate the model's performance, we conducted experiments under two settings: one that incorporated user bias and one that did not. For each positive sample in the training data (i.e., a purchased product), we generated a negative sample (i.e., a randomly unpurchased product). Training continued until the loss stabilized.

We conducted the evaluation using two widely adopted metrics:

- **NDCG@k (Normalized Discounted Cumulative Gain)** evaluates both the relevance and ranking position of recommended items. We report NDCG@5 and NDCG@10 to measure the quality of the top five and top ten recommendations, respectively.
- **HR@k (Hit Rate)** is the proportion of recommended items with which the user engaged among the top k items. We used HR@5 and HR@10 to evaluate the model's ability to recall recommendations for lists of different sizes.

Combining NDCG and HR provides a more comprehensive evaluation of the recommender system. NDCG focuses on ranking quality, while HR evaluates coverage and alignment with user interests. Together, these metrics offer valuable insights into the system's accuracy, relevance, and user experience.

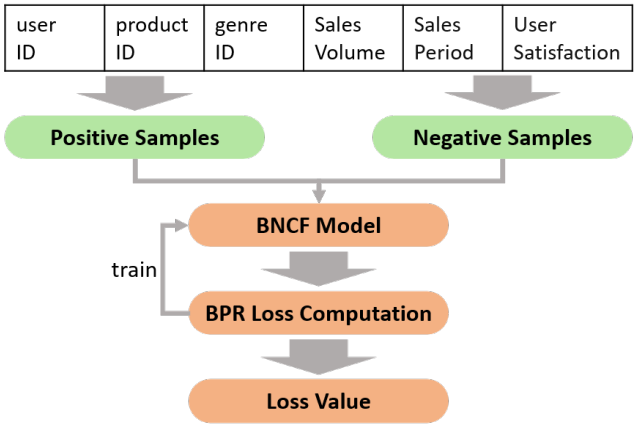


Figure 5. BNCF model training flowchart.

4. Model Architecture Analysis and Evaluation

In this section, we conducted a systematic evaluation of the proposed model's architecture. The evaluation proceeded in two phases. First, we adjusted the model's structure and parameters and then trained it using a designated training dataset. In the second phase, we evaluated the trained models using a test dataset and conducted a comparative analysis to identify the most effective architecture. First, we implemented a baseline model without item bias and compared it to an extended model that incorporated item bias factors. Based on the results, we experimented with varying the number of negative samples, the number of hidden layers, and the dimensionality of the embedding vectors. We applied these steps to the training and testing procedures to select the optimal model configuration.

4.1. Dataset and Preprocessing

The dataset used in this study was sourced from the Rakuten Ichiba data available in the "Rakuten Dataset," provided by Rakuten Group, Inc. through the IDR Dataset Service of the National Institute of Informatics (NII) [30]. The dataset spans the entirety of 2019 and contains monthly records across five product categories: bags, home appliances, cosmetics, women's fashion, and shoes.

For each product, we extracted relevant information, including product name, product ID, genre ID, user rating, review content, and review timestamp. The dataset also includes user ID and product-level metadata, such as the number of reviews. We applied data cleaning procedures to remove duplicates, null values, and improperly formatted entries.

We constructed two distinct subsets of the dataset to evaluate the two proposed recommendation approaches:

- **Cold-start recommendation (for users without purchase histories):** We selected a sample of 300 products and approximately 18,000 reviews. This subset was used to calculate product-level recommendation scores based on sentiment, sales volume, and sales period, as described in Section 3.1.
- **Personalized recommendation (for users with prior interactions):** We filtered the dataset to include only users who had interacted with at least two products and products purchased by at least two users. The resulting interaction matrix included 4570 users, 4774 products, and 16,241 user–item interactions. To evaluate the BNCF model, we split the dataset by user into training and test sets. One interaction was ran-

domly selected per user for the test set (706 users), while the remaining were used for training (3723 users). The result is 11,057 training interactions and 706 test interactions.

4.2. Experimental Settings

4.2.1. Experimental Environment

All experiments were conducted on a consistent hardware and software platform to ensure the reproducibility of our results. The specific settings for our experimental environment are detailed in Table 2.

Table 2. Experimental hardware and software configuration.

Category	Component	Specification
Hardware	CPU	Intel Core i7-12700H @ 2.30 GHz (14 cores)
	GPU	NVIDIA GeForce RTX 3060 (6GB VRAM)
	Memory (RAM)	16 GB DDR4
Software	Operating System	Windows 11 Home
	Programming Language	Python 3.12.9
	Deep Learning Framework	PyTorch 2.5.0+cu124
	Data Manipulation	Pandas 1.4.2
	Scientific Computing	NumPy 1.21.5
	NLP/Sentiment	Google Cloud Natural Language API

4.2.2. Hyperparameter Tuning

The BNCF model was initialized with random parameters and optimized using the Adam optimizer [29]. To determine the optimal configuration, we conducted hyperparameter tuning through a grid search strategy. The validation set was created by holding out the last interaction of each user from the training data. Training and validation loss during the model training for 200 epochs.

Table 3 presents the search ranges for each hyperparameter along with the optimal values selected. The final configuration includes an embedding size of 32, 4 negative samples per positive, 2 MLP layers, a learning rate of 0.001, a mini-batch size of 256, and a weight decay of 0.00001.

Table 3. BNCF hyperparameter tuning ranges and optimal values.

Hyperparameter	Search Range	Optimal Value
Learning Rate	{0.0001, 0.001, 0.01}	0.001
Embedding Size	{32, 64, 128}	32
Number of Negative Samples per Positive	{1, 4, 8, 16}	4
Number of Layers	{1, 2, 3}	2
Mini-batch Size	{256}	256
Weight Decay (L2)	{0, 0.00001, 0.0001, 0.001}	0.00001
Epochs	N/A	200

4.2.3. Loss Function and Training Sample Generation

For model training, we adopted the BPR loss, a pairwise objective function well-suited for implicit feedback datasets. We employed the standard BPR triplet sampling strategy for training both the NCF and BNCF models.

For each observed user–item interaction (u, i), which is treated as a positive sample, we randomly selected one or more items j from the set of items that user u has not interacted with. Each training instance thus forms a triplet (u, i, j), and the model is optimized to maximize the probability that user u's predicted preference score for positive item i exceeds that for negative item j. This negative sampling was performed uniformly and without

replacement from the user's set of non-interacted items. The sampling process was repeated dynamically in each training epoch to provide the model with diverse negative examples over time. The number of negative samples generated per positive instance was controlled as a tunable hyperparameter, with the optimal value detailed in our hyperparameter settings (see Table 3).

4.3. Accuracy Verification of Product Recommendation Rankings for New Users Without Purchase Histories

1. **Data Processing**
   We used the same dataset described in Section 4.1, which includes review data for the entire year of 2019. We selected five main product categories, each comprising three subcategories. Within each subcategory, we classified products based on the number of reviews and selected the top 20 products from each subcategory, resulting in a total of 300 products. Next, we performed sentiment analysis on the review texts and calculated three key scores for each product: sales volume score, sales period score, and user satisfaction score. These scores were then binarized ("high" or "low") using threshold-based analysis and normalized. An overall score was subsequently computed to rank products within each subcategory, facilitating the classification and selection of products for further evaluation.
2. **Experimental Method**
   We randomly selected one subcategory from each of the five product categories. Then, we randomly selected 20 products from each subcategory, for a total of 100 products. To evaluate the accuracy of the top ten recommended product rankings, we surveyed twelve university students using a questionnaire. Each participant received comprehensive product information, including the product name, ID, genre ID, user rating, review title and text, sales volume, and sales period. After reviewing this information and the associated reviews, the participants selected their ten preferred products from each subcategory. This questionnaire did not require ethics committee approval as it did not involve animal or human clinical trials and did not raise significant ethical concerns. This study adhered to the ethical principles outlined in the Declaration of Helsinki. All participants were informed of the questionnaire's purpose and provided their informed consent before participating. All participants were guaranteed anonymity and confidentiality, and participation was entirely voluntary. Each product received at least six evaluations. To measure ranking accuracy, we employed three widely used ranking metrics: mean reciprocal rank (MRR), mean average precision (MAP), and normalized discounted cumulative gain at rank 10 (NDCG@10). We computed these metrics using the participants' rankings as the ground truth.
3. **Experimental Results**
   As shown in Table 4, the average MRR across all subcategories is 0.323. The highest MRR is observed in the "Dryer" category (0.450), while the lowest is in "Sweater" (0.142), suggesting suboptimal accuracy for specific product types. The average MAP score is 0.618, with "Suitcase" achieving the highest (0.676) and "Essential Oil" the lowest (0.572), indicating moderate precision in ranking. The average NDCG@10 score is 0.567. The highest score is in the "Suitcase" category (0.623), and the lowest scores are in the "Essential Oil" and "Leather Shoes" categories (0.516 and 0.517, respectively), showing relatively acceptable top-10 ranking performance.

Table 4. Ranking accuracy results by rank-aware metrics: MRR, MAP, and NDCG@10.

Category	MRR	MAP	NDCG@10
Suitcase	0.348	0.676	0.623
Sweater	0.142	0.614	0.586
Dryer	0.450	0.637	0.593
Essential Oil	0.263	0.572	0.516
Leather Shoes	0.431	0.590	0.517
AVE	0.323	0.618	0.567

4.4. Comparison of NCF Model with and Without Bias

To evaluate the effectiveness of incorporating item bias into the proposed BNCF model, we compared it to a baseline NCF model that lacked bias factors. All experiments used 32-dimensional embedding vectors, a 64-dimensional hidden layer, and a 32-dimensional output layer. To ensure a fair comparison, we held the BPR loss function and other hyperparameters constant across models. Each model was trained for 200 epochs. We performed evaluations using NDCG and HR at k = 5 and k = 10. Table 5 summarizes the results.

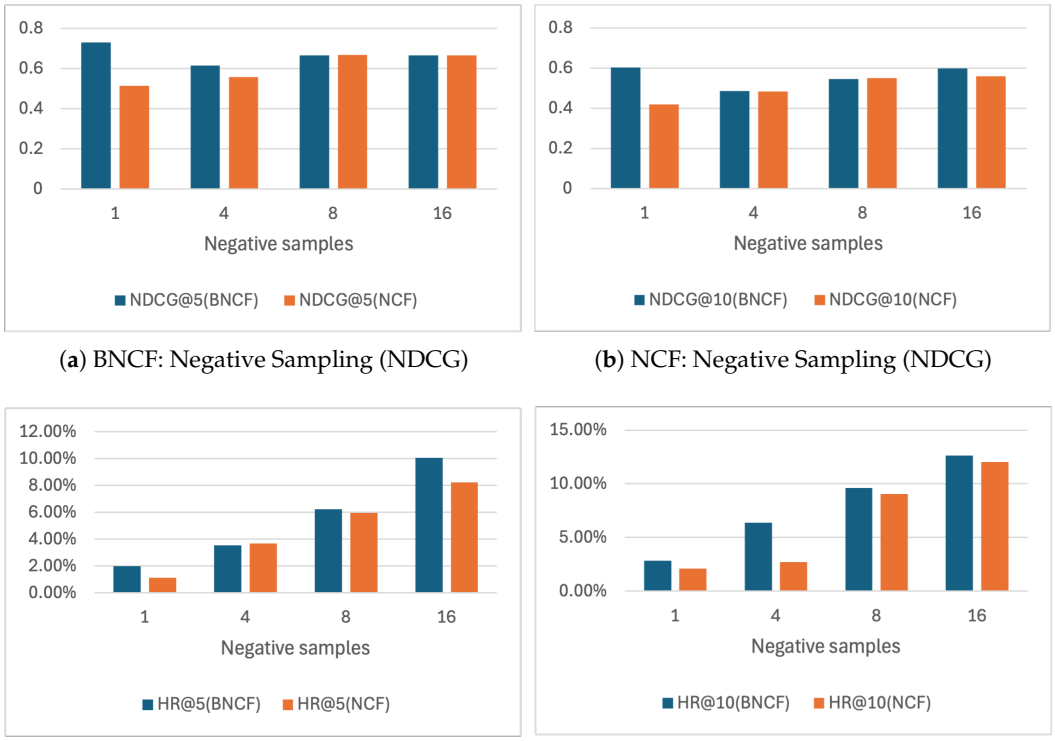
Table 5. Performance comparison of NCF with and without bias across varying the number of negative samples.

Bias	Negative Samples	NDCG@5	NDCG@10	HR@5	HR@10
With	1	0.7293	0.6023	1.98%	2.83%
	4	0.6143	0.4856	3.54%	6.37%
	8	0.6652	0.5444	6.23%	9.63%
	16	0.6645	0.5981	10.06%	12.61%
Without	1	0.5136	0.4200	1.13%	2.12%
	4	0.5576	0.4841	3.68%	2.69%
	8	0.6685	0.5500	5.95%	9.07%
	16	0.6648	0.5590	8.22%	12.04%

As shown in Figure 6, increasing the number of negative samples improves both NDCG and HR metrics for both models. This improvement stems from the model's enhanced ability to learn from more negative examples, which strengthens its generalization capability. However, using too many negative samples can significantly increase training time and the risk of overfitting. It highlights the importance of balancing performance with computational cost, which is crucial for achieving optimal efficiency and reducing resource consumption.

The BNCF model outperforms the NCF model when negative samples are scarce. It is evident from its higher NDCG@5 and NDCG@10 scores. It suggests that incorporating bias factors provides contextual information, enabling the model to capture latent user–item interactions accurately. As the number of negative samples increases, however, the performance gap between the two models narrows, although the BNCF model still performs slightly better.

On the other hand, the unbiased NCF model shows more consistent improvement with negative samples. It suggests that it relies more on interaction data. Larger sample sizes allow the model to more accurately identify user preferences, thereby improving hit rates and ranking quality. However, in this scenario, the relative contribution of item bias decreases while model complexity increases, which could lead to overfitting.



(**a**) BNCF: Negative Sampling (NDCG) (**b**) NCF: Negative Sampling (NDCG)

(**c**) BNCF: Negative Sampling (HR) (**d**) NCF: Negative Sampling (HR)

Figure 6. Comparison of NDCG and HR metrics for BNCF and NCF models across different numbers of negative samples (1, 4, 8, and 16).

Therefore, we recommend using the BNCF model when the number of negative samples is limited as the bias factors significantly improve performance. For large-scale negative sampling with sufficient computational resources, the simpler NCF model without bias can maintain competitive performance while reducing the risk of overfitting and training overhead.

4.5. Comparison with Baseline Models

To validate the effectiveness of our proposed BNCF model, we conducted a comparative evaluation against three baseline models: the standard NCF, a classic model with explicit bias terms (BiasedMF) [16], and a representative state-of-the-art model based on graph neural networks (LightGCN) [31].

- **NCF:** This model serves as our foundational baseline. Since it does not incorporate any form of bias modeling, it is used to isolate the performance gains achieved by our proposed bias integration.
- **BiasedMF:** It is a classical matrix factorization model that incorporates scalar user and item bias terms. This model provides a basis for evaluating the effectiveness of explicit bias modeling in traditional recommender systems.
- **LightGCN:** This model represents a state-of-the-art approach based on graph convolutional networks. It captures high-order connectivity within the user–item interaction graph and is widely recognized for its strong performance in recommendation tasks.

All baseline models were implemented using publicly available libraries. Their hyperparameters were optimized using our validation set to ensure a fair and meaningful comparison.

As shown in Table 6, the standard NCF model achieves an NDCG@10 of 0.6209 and an HR@10 of 8.07%. Its lack of bias modeling limits its ability to personalize recommendations

based on item-specific factors. While BiasedMF introduces explicit bias factors, it relies on linear interactions, and it cannot capture complex user–item dynamics. Consequently, its performance is substantially lower. The NDCG@10 is 0.4779. The HR@10 is only 1.98%.

Table 6. Performance comparison of BNCF and baseline models.

Model	NDCG@10	HR@10	Description
NCF	0.6209	8.07%	Base model without any bias component
BiasedMF	0.4779	1.98%	Classic model with explicit bias terms
LightGCN	0.9721	19.73%	State-of-the-art GNN-based model
BNCF (ours)	0.9848	19.69%	Proposed model with multi-faceted bias

In contrast, LightGCN uses the user–item graph structure and propagates latent preferences through multiple graph convolution layers. This model demonstrates strong overall performance, achieving an NDCG@10 of 0.9721 and an HR@10 of 19.73%.

Our proposed BNCF model combines the expressiveness of deep neural networks with domain-aware bias integration. By incorporating normalized sales volume, sales period, and user satisfaction scores into the embedding space, the BNCF model effectively captures both business-driven signals and nonlinear user–item interactions. The model achieves the highest NDCG@10 of 0.9848, indicating superior ranking precision. Its HR@10 reaches 19.69%, which is comparable to the performance of LightGCN.

These results confirm the advantage of combining multidimensional bias information with deep collaborative filtering frameworks. The BNCF model provides personalized accurate recommendations, outperforming traditional matrix factorization methods and advanced graph-based models in terms of ranking accuracy.

4.6. Ablation Study of Bias Components

To assess the individual contribution of each proposed bias component in the BNCF model, we conducted an ablation study. Specifically, we created three variants of the model by selectively removing one bias factor at a time. These factors include sales volume (Vol), sales period (Per), and user satisfaction (Sat). Each variant retains the remaining two components. For comparative purposes, we also included the standard NCF model, which excludes all bias factors entirely.

Each model variant was trained under the identical configuration detailed in Section 4.2, with a mini-batch size of 256 and 4 negative samples per positive instance. The performance results, evaluated using NDCG@10 and HR@10, are presented in Table 7.

Table 7. Ablation study results of bias components.

Bias Configuration	NDCG@10	HR@10
Full BNCF (Vol, Per, Sat)	0.9848	19.69%
Without Satisfaction (Vol, Per)	0.9773	19.83%
Without Sales Period (Vol, Sat)	0.9937	19.41%
Without Sales Volume (Per, Sat)	0.9831	19.41%
No Bias (Standard NCF)	0.6209	8.07%

The results in Table 7 provide valuable insights into the role of each bias component. All model variants that incorporate bias factors outperform the standard NCF model. These results confirm the effectiveness of incorporating product-level attributes into the recommendation process.

Among the three components, sales volume (Vol) has the strongest influence on hit rate. Removing this factor causes HR@10 to decline from 19.69% to 19.41%. This observa-

tion indicates that product popularity plays a critical role in enhancing user engagement. In contrast, user satisfaction (Sat) has the most significant effect on ranking precision. Its removal results in a decrease in NDCG@10 from 0.9848 to 0.9773, while HR@10 slightly increases. It suggests a shift toward recommending more popular but potentially less relevant items.

The sales period (Per) component exhibits the least impact on both metrics. Its removal leads to a slight increase in NDCG@10, suggesting that this feature may provide limited additional value when sales volume (Vol) and user satisfaction (Sat) are already present.

In summary, this ablation study shows that the full BNCF model provides the most balanced and effective performance. Sales volume (Vol) contributes to exposure and engagement, while user satisfaction (Sat) enhances personalized ranking. The combination of these diverse factors enables the model to provide recommendations that are both accurate and user-centered.

4.7. Analysis of Hidden Layer Structures in BNCF Model

To enhance the model's capacity for capturing intricate user–item interactions, we investigated the impact of various hidden layer architectures on the performance of the BNCF model. Specifically, we modified the original NCF model by introducing additional hidden layers to evaluate their effect. We defined the baseline architecture (Model A) as 32–64–32. We created three enhanced variants: Model B had one additional hidden layer, and Model C had two additional hidden layers. The bias component structure remained unchanged across all models.

These modifications were made based on the idea that deeper architectures can better capture complex nonlinear interactions, which could lead to richer feature representations and improved generalization. To analyze the interaction between architecture depth and negative sampling, we evaluated each model using 1, 4, and 8 negative samples.

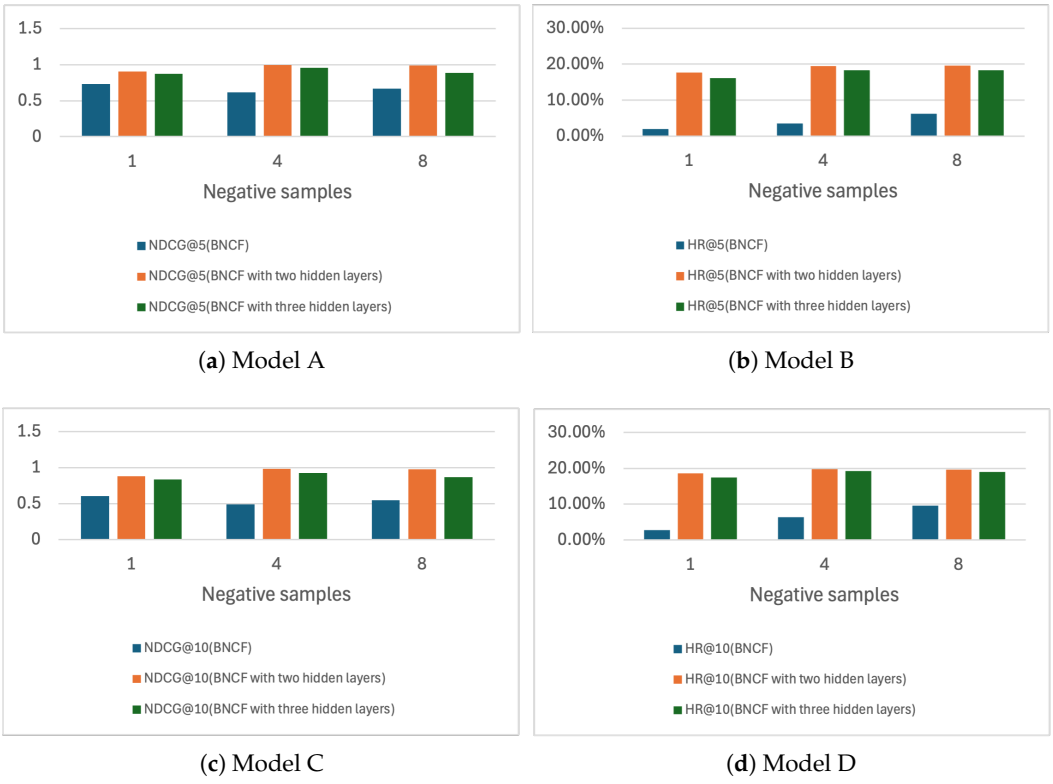
As shown in Table 8, increasing the model depth results in clear performance gains. For example, Model B achieved significant improvements over Model A when using one negative sample: NDCG@5 and NDCG@10 increased by 24.31% and 46.14%, respectively. In terms of hit rate, HR@5 and HR@10 increased approximately eightfold and 5.56-fold, respectively. These results suggest that the added hidden layer in Model B significantly enhances the model's expressiveness.

Table 8. Performance metrics of different models.

Model	Negative Samples	NDCG@5	NDCG@10	HR@5	HR@10
A	1	0.7293	0.6023	1.98%	2.83%
	4	0.6143	0.4856	3.54%	6.37%
	8	0.6652	0.5444	6.23%	9.63%
B	1	0.9066	0.8802	17.71%	18.56%
	4	0.9946	0.9848	19.41%	19.69%
	8	0.9884	0.9742	19.55%	19.67%
C	1	0.8744	0.8334	16.15%	17.42%
	4	0.9560	0.9234	18.27%	19.26%
	8	0.8886	0.8675	18.27%	18.98%

As shown in Figure 7, increasing the number of negative samples from 1 to 4 substantially improves performance. However, the marginal benefit diminishes when the number of negative samples increases from 4 to 8. Furthermore, models with excessive depth may encounter training instability. For example, Model C exhibited numerical instability during loss computation, particularly at higher negative sampling rates. This predicament

is ubiquitous in deep networks as vanishing or exploding gradients can compromise the integrity of the training process. As suggested by Kloberdanz et al. [32], introducing a small epsilon (e.g., 1×10^{-9}) into the logarithmic function can mitigate such numerical issues, particularly avoiding undefined operations like $\log(0)$.



(**a**) Model A (**b**) Model B

(**c**) Model C (**d**) Model D

Figure 7. Performance of different models under various negative sampling conditions.

Among the three variants, Model B consistently delivers the best overall performance, especially when using four or more negative samples. Model C, while deeper, does not provide significant performance gains over Model B and introduces greater computational and numerical complexity. Model A, as the baseline, shows the weakest performance.

These results indicate that Model B (32–64–64–32) offers an effective trade-off between expressiveness and computational efficiency. Its moderate complexity allows it to model nonlinear user–item interactions effectively without incurring the training difficulties observed in deeper models. Moreover, using four negative samples provides a practical balance between recommendation performance and computational cost. Based on these findings, we conclude that Model B with four negative samples is the optimal configuration for our BNCF model.

4.8. Analysis of Embedding Dimensions in BNCF Model

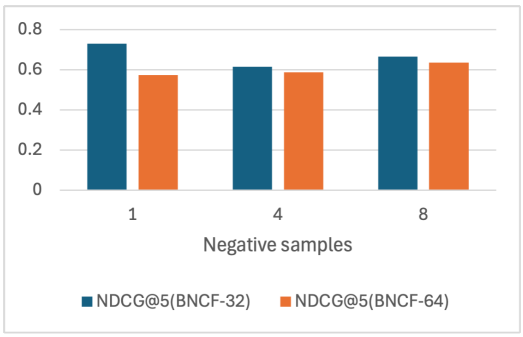
We investigated how the size of the embedding dimension affects the performance of the BNCF model. The original model, BNCF-32, used embeddings with dimensions of 32, 64, and 32. Next, we constructed a larger variant, BNCF-64, with dimensions of 64, 128, and 64. Both models included item bias terms of equal size. We conducted experiments using 1, 4, and 8 negative samples.

As illustrated in Table 9 and Figure 8, increasing the embedding dimension did not lead to performance improvement. BNCF-32 outperformed BNCF-64 across all evaluation metrics. For example, when using one negative sample, BNCF-32 achieved significantly higher NDCG@5 (0.7293 vs. 0.5725) and HR@5 (1.98% vs. 1.27%). Even when using

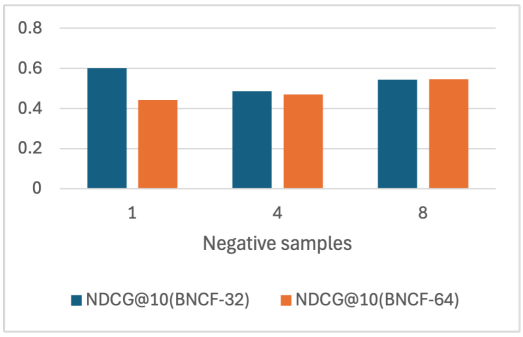
eight negative samples, the improvement in BNCF-64 remained marginal and did not surpass BNCF-32.

Table 9. Performance metrics of different BNCF models.

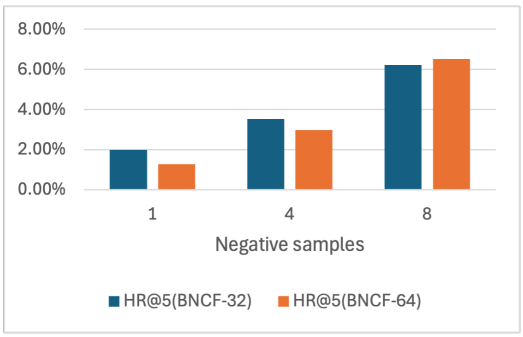
Model	Negative Samples	NDCG@5	NDCG@10	HR@5	HR@10
BNCF-32	1	0.7293	0.6023	1.98%	2.83%
	4	0.6143	0.4856	3.54%	6.37%
	8	0.6652	0.5444	6.23%	9.63%
BNCF-64	1	0.5725	0.4430	1.27%	2.69%
	4	0.5881	0.4709	2.97%	5.24%
	8	0.6359	0.5454	6.52%	9.21%



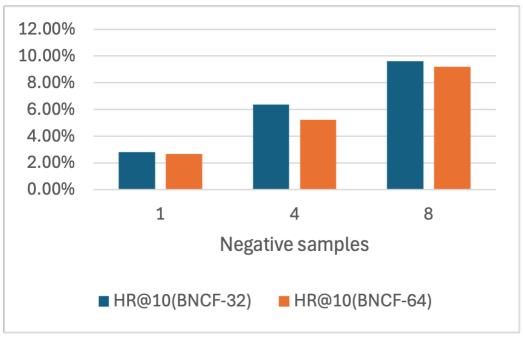
(**a**) NDCG (BNCF-32)



(**b**) NDCG (BNCF-64)



(**c**) HR (BNCF-32)



(**d**) HR (BNCF-64)

Figure 8. Performance comparison of BNCF-32 and BNCF-64 across NDCG and HR metrics.

Several factors may explain this degradation. First, increasing embedding dimensions substantially enlarges the parameter space, which raises the risk of overfitting, particularly in scenarios with limited training data. Second, complex models are more sensitive to noise and require more robust regularization. Third, deep models with large embeddings are susceptible to vanishing or exploding gradients, which complicates the training process and can lead to suboptimal convergence.

In contrast, BNCF-32 provides a well-balanced configuration: its complexity is sufficient to capture meaningful user–item interaction patterns without introducing the instability and overfitting risks associated with larger embeddings. The results suggest that further enlarging embedding dimensions beyond a certain threshold does not guarantee improved performance and may hinder generalization.

In conclusion, the original BNCF-32 architecture demonstrates superior robustness and efficiency compared to its higher-dimensional counterpart. It effectively balances representational power, training stability, and computational cost, making it the preferred choice for practical applications.

5. Discussion

In Sections 3.1 and 3.2, we introduced two distinct recommendation methods. Despite the differences in dataset composition and experimental design, a comparison of the NDCG metrics allows us to assess and contrast their effectiveness. We refer to the recommendation approach for users with no purchase histories as Method 1 and the approach tailored for users with prior purchase behavior as Method 2.

5.1. Method 1: Rule-Based Recommendation for Cold-Start Users

In Method 1, we focused on five product categories: suitcases, sweaters, dryers, essential oils, and leather shoes, selecting 20 items per category for a total of 100 products. Recommendations were generated by ranking the products based on a composite score derived from sales volume, sales duration, and user satisfaction.

The experimental results showed an average NDCG@10 of 0.567, with the suitcase category achieving the highest score (0.623). Categories such as essential oils (0.516) and leather shoes (0.517) exhibited lower performance. Upon analysis, we attribute this disparity to the uniform weighting of the factors in our scoring function. Although the method assumes that users value sales volume, sales period, and satisfaction equally, user preferences vary significantly. Some users prioritize longevity in sales (sales period), while others focus more on recent popularity (sales volume) or product quality (user satisfaction). This variability leads to a divergence between system-generated rankings and user-perceived relevance.

For example, a product with a high sales volume, a short sales duration, and high customer satisfaction is likely to be popular and widely accepted. These attributes attract users. However, if the model continues to use a static weighting formula, it may misrepresent user intent, resulting in less precise recommendations. Evaluation metrics such as MAP and NDCG@10 reflect this issue. These metrics typically range between 0.5 and 0.6, suggesting that nearly half of recommendations deviate from user preferences.

5.2. Method 2: Deep Learning-Based BNCF for Personalized Recommendation

In Method 2, we employed a larger and more complex dataset consisting of 4570 users, 4774 items, and 16,241 records of user–item interactions. We proposed the BNCF model, which integrates three item-level bias features—sales volume, sales period, and user satisfaction—into the standard NCF architecture. After testing, we randomly selected the bias embedding vector parameters of 20 users from the model that showed the best performance and mapped them to the 0–1 interval using the sigmoid function. Figure 9 shows the relationship between the three bias parameters of the users.

Under different configurations of hidden layers and negative sampling rates (1, 4, and 8), the model demonstrated strong performance. For instance, with one negative sample per user, NDCG@10 reached 0.8802; with four negative samples, it increased further to 0.9848. These results are markedly superior to those obtained in Method 1.

Several factors contributed to this performance improvement:

1. **Rich and Diverse Dataset:**
   Method 2 benefits from a significantly larger dataset, allowing the model to learn more nuanced patterns of user preferences and item interactions, which improves the generalization and accuracy of recommendations.
2. **Advanced Model Architecture:**
   Method 2 uses a neural collaborative filtering framework to capture the complex nonlinear interactions between users and items. It is a significant advantage over Method 1's rule-based approach as it cannot adapt to intricate user behavior.
3. **Integration of Bias Features:**

The inclusion of item-specific bias factors enhanced the model's contextual understanding of products. By embedding attributes such as popularity and user satisfaction directly into the model, the BNCF approach provided a richer and more personalized recommendation output.

Collectively, these factors illustrate the importance of employing sophisticated models and multidimensional item data to improve the quality of recommendations.

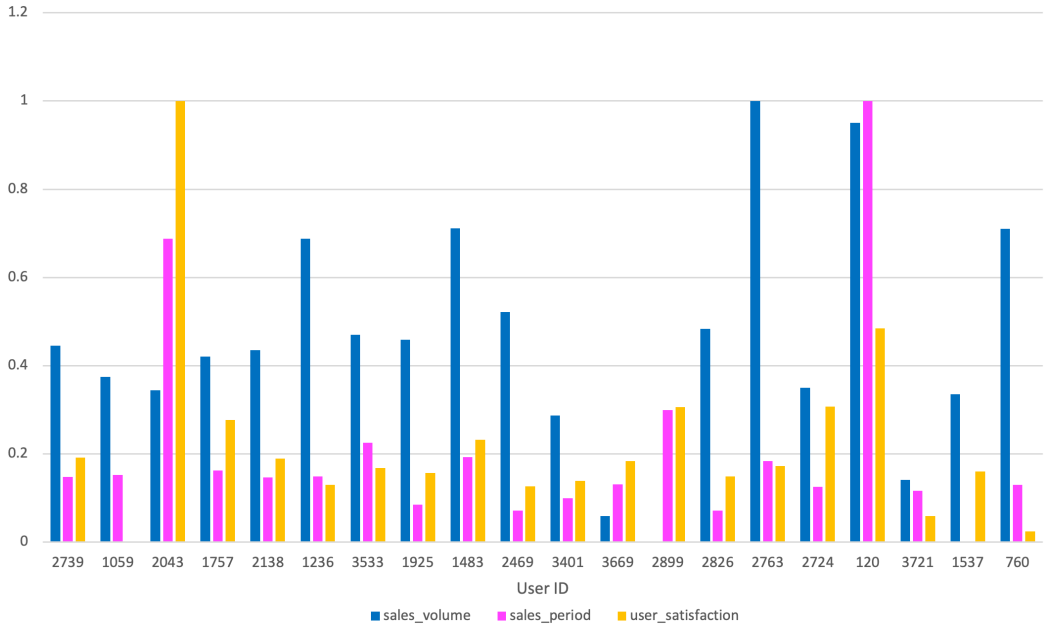


Figure 9. Bias parameter visualization of BNCF model.

5.3. Comparative Model Analysis Within Method 2

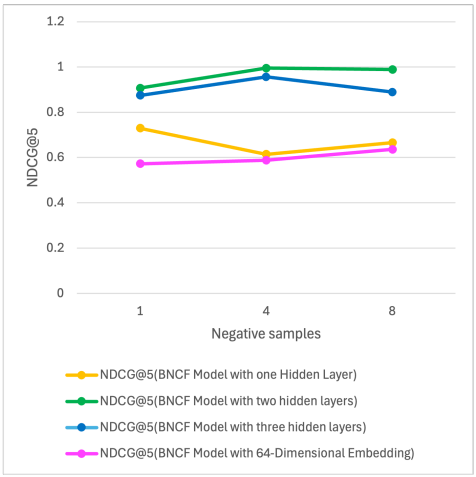
To further refine Method 2, we conducted comparative experiments by varying the architecture of the BNCF model. Specifically, we tested different hidden-layer configurations and embedding dimensions. The goal was to identify an optimal balance between model complexity, expressive capacity, and generalization performance.

Among the configurations tested, the 32–64–64–32 architecture consistently outperformed the others, as indicated in Figure 10. This model achieved the best overall performance across both the NDCG and HR metrics. Compared to the simpler 32–64–32 structure or the larger 64–128–64 configuration, this architecture provided enhanced feature representation without incurring significant computational overhead or overfitting.

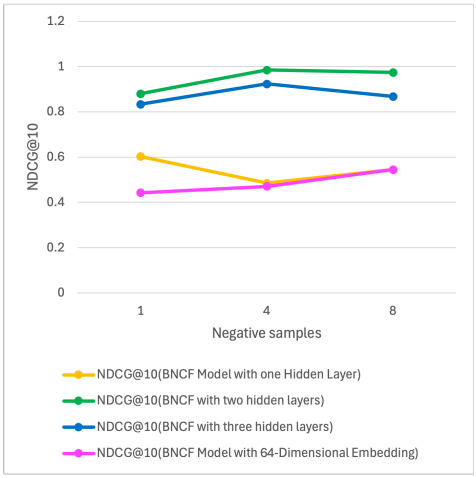
Our experiments also underscored the importance of the following:

- **Managing Numerical Stability:** During training, we encountered instability issues in deeper models (e.g., gradient explosion/vanishing). Mitigating these through regularization techniques and loss function smoothing (e.g., adding small constants to avoid $\log(0)$) proved essential for stable optimization.
- **Selecting Optimal Negative Sampling Ratios:** We found that increasing the number of negative samples to four substantially improved performance. However, further increases yielded diminishing returns and increased computational cost. Thus, using four negative samples offered a practical trade-off between performance and efficiency.

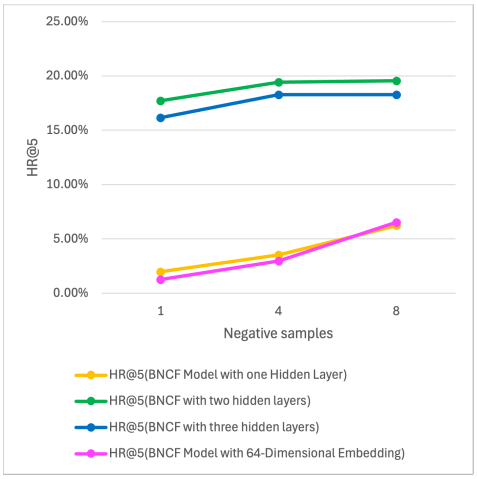
In summary, Method 2 demonstrates that thoughtful architectural design, feature integration, and training strategies can dramatically enhance recommendation performance. In real-world applications, the combination of deep learning techniques, multidimensional item features, and user interaction data offers a robust solution to the challenges faced by modern recommender systems.



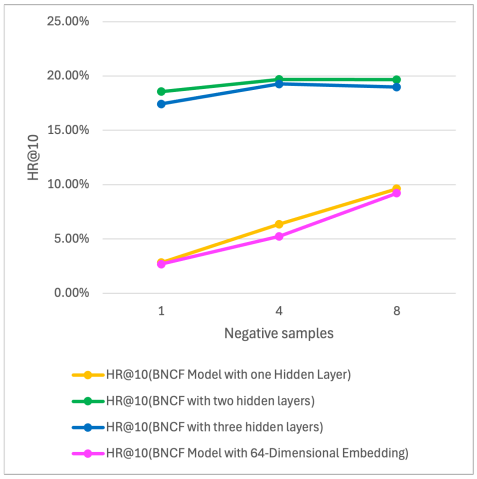
(**a**) NDCG (BNCF-32)



(**b**) NDCG (BNCF-128)



(**c**) HR (BNCF-32)



(**d**) HR (BNCF-128)

Figure 10. Comparison of performance metrics (NDCG and HR) for BNCF models with different structures: BNCF-32 and BNCF-128.

6. Conclusions

In this study, we proposed two recommendation strategies tailored to different user scenarios: one for users without purchase histories and another for users with prior interactions. For users without purchase histories, we developed a rule-based recommendation method grounded in three key product attributes: sales volume, sales period, and user satisfaction. We quantified user satisfaction through sentiment analysis combined with rating scores. Then, we derived a composite score through threshold analysis to rank products. We evaluated the system's effectiveness using product review data from Rakuten Ichiba. The results showed that the method could identify and recommend products with high user satisfaction, strong sales, and extended sales cycles. However, it failed to align with individual user preferences due to the lack of personalized behavioral data.

To address this limitation, we proposed a BNCF model for users with purchase histories. The BNCF model extends the standard neural collaborative filtering framework by incorporating item-level bias factors, specifically sales volume, sales period, and user satisfaction, into the recommendation process. We conducted extensive experiments to examine the impact of various conditions, including the inclusion of bias factors, different numbers of negative samples, variations in model depth, and embedding dimensionality. These experiments led to the identification of an optimal BNCF configuration, which

demonstrated superior performance in both ranking quality and accuracy compared to the baseline models.

The comparative analysis of the two methods provided several key insights. Users without purchase histories present a significant challenge due to the absence of interaction data, which restricts the model to general recommendations based solely on product-level attributes. As a result, the capacity to deliver personalized and precise recommendations is considerably limited. In contrast, users with purchase histories provide valuable behavioral data that we can use to model intricate user–item interactions. The BNCF model uses deep learning techniques. These techniques identify latent relationships. The result is more precise and customized recommendations. The experimental results confirmed that this approach significantly improves recommendation accuracy and user satisfaction.

Future research will aim to expand the dataset and improve product classification granularity for users without purchase histories. We plan to represent product features within a three-dimensional space and introduce a dynamic weighting mechanism to account for variations in user preference priorities.

For users with prior interactions, future efforts will focus on enhancing the BNCF framework through the integration of the following techniques:

- Graph neural networks (GNNs) can model complex relationships among users and items across different types of interactions.
- Cross-modal attention mechanisms effectively fuse multiple data sources, including text, images, and numbers.
- Residual connections can stabilize training and enhance the performance of deeper network architectures.

These advancements are expected to improve the accuracy, robustness, and adaptability of recommender systems in real-world applications.

Author Contributions: Conceptualization, H.X. and Y.W.; methodology, H.X.; software, H.X.; validation, H.X. and Y.W.; formal analysis, H.X.; investigation, H.X.; resources, H.X.; data curation, H.X.; writing—original draft preparation, H.X.; writing—review and editing, H.X. and Y.W.; visualization, H.X.; supervision, Y.W.; project administration, Y.W. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Ethical review and approval were waived for this study due to its nature as an anonymous user questionnaire survey, which does not involve personal data collection or invasive procedures, in accordance with the "Ethical Guidelines for Medical and Biological Research Involving Human Subjects" established by the Japanese Ministry of Health, Labour, and Welfare (MHLW) and the "Regulations for General Research Involving Human Subjects" at Yamaguchi University.

Informed Consent Statement: Informed consent was obtained from all subjects involved in the study.

Data Availability Statement: The data analyzed in this study is subject to the following licenses and restrictions: National Institute of Informatics's Informatics Research Data Repository for Research Purposes. Requests to access this dataset should be directed to https://rit.rakuten.com/data_release/ (accessed on 21 May 2023). Further inquiries can be directed to the corresponding authors.

Acknowledgments: In this paper, the authors used the "Rakuten Dataset" (https://rit.rakuten.com/data_release/) (accessed on 21 May 2023) provided by Rakuten Group, Inc. via IDR Dataset Service of the National Institute of Informatics.

Conflicts of Interest: Author Haoyang Xia is employed by MICWARE Co., Ltd. However, this research was conducted independently as part of his doctoral studies at Yamaguchi University and was not supported by MICWARE Co., Ltd. The authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as potential conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

BiasedMF	Biased matrix factorization
BNCF	Bias neural collaborative filtering
BPR	Bayesian personalized ranking
CF	Collaborative filtering
CNNs	Convolutional neural networks
DeepCoNNs	Deep cooperative neural networks
DNNs	Deep neural networks
GCNs	Graph convolutional networks
GNNs	Graph neural networks
HR	Hit rate
KNN	K-nearest neighbor
LSTM	Long short-term memory
MAP	Mean average precision
MRR	Mean reciprocal rank
NDCG	Normalized discounted cumulative gain
NCF	Neural collaborative filtering
NII	National Institute of Informatics
NLP	Natural language processing
RC-DFM	Review and content-based deep fusion model
ReLU	Rectified linear unit

References

1. Lackermair, G.; Kailer, D.; Kanmaz, K. Importance of online product reviews from a consumer's perspective. *Adv. Econ. Bus.* **2013**, *1*, 1–5. [CrossRef]
2. Changchit, C.; Klaus, T. Determinants and impact of online reviews on product satisfaction. *J. Internet Commer.* **2020**, *19*, 82–102. [CrossRef]
3. Im, I.; Hars, A. Does a one-size recommendation system fit all? The effectiveness of collaborative filtering based recommendation systems across different domains and search modes. *ACM Trans. Inf. Syst. (TOIS)* **2007**, *26*, 4-es. [CrossRef]
4. Çano, E.; Morisio, M. Hybrid recommender systems: A systematic literature review. *Intell. Data Anal.* **2017**, *21*, 1487–1524. [CrossRef]
5. Sarwar, B.; Karypis, G.; Konstan, J.; Riedl, J. Item-based Collaborative Filtering Recommendation Algorithms. In Proceedings of the WWW'01: 10th International Conference on World Wide Web, Hong Kong, China, 1–5 May 2001; pp. 285–295. [CrossRef]
6. Ko, H.; Lee, S.; Park, Y.; Choi, A. A Survey of Recommendation Systems: Recommendation Models, Techniques, and Application Fields. *Electronics* **2022**, *11*, 141. [CrossRef]
7. Bobadilla, J.; Dueñas-Lerín, J.; Ortega, F.; Gutierrez, A. Comprehensive evaluation of matrix factorization models for collaborative filtering recommender systems. *Int. J. Interact. Multimed. Artif. Intell.* **2024**, *8*, 15–23. [CrossRef]
8. Adeniyi, D.; Wei, Z.; Yongquan, Y. Automated web usage data mining and recommendation system using K-Nearest Neighbor (KNN) classification method. *Appl. Comput. Inform.* **2016**, *12*, 90–108. [CrossRef]
9. Zou, C.; Zhang, D.; Wan, J.; Hassan, M.M.; Lloret, J. Using concept lattice for personalized recommendation system design. *IEEE Syst. J.* **2017**, *11*, 305–314. [CrossRef]
10. Wu, J.; Rao, H.; Fan, X.; Wang, Y. A multidimensional application recommender system based on user feature clustering and contextual information. *J. Integr. Technol.* **2021**, *10*, 22. [CrossRef]
11. He, X.; Liao, L.; Zhang, H.; Nie, L.; Hu, X.; Chua, T.S. Neural Collaborative Filtering. In Proceedings of the WWW'17: 26th International Conference on World Wide Web, Perth, Australia, 3–7 April 2017; pp. 173–182. [CrossRef]

12. Messaoudi, F.; Loukili, M. E-commerce personalized recommendations: A deep neural collaborative filtering approach. *SN Oper. Res. Forum* **2024**, *5*, 5. [CrossRef]
13. Cheng, H.T.; Koc, L.; Harmsen, J.; Shaked, T.; Chandra, T.; Aradhye, H.; Anderson, G.; Corrado, G.; Chai, W.; Ispir, M.; et al. Wide & Deep Learning for Recommender Systems. In Proceedings of the DLRS 2016: 1st Workshop on Deep Learning for Recommender Systems, Boston, MA, USA, 15 September 2016; pp. 7–10. [CrossRef]
14. Patel, R.; Thakkar, P.; Ukani, V. CNNRec: Convolutional neural network based recommender systems—A survey. *Eng. Appl. Artif. Intell.* **2024**, *133*, 108062. [CrossRef]
15. Daneshvar, H.; Ravanmehr, R. A social hybrid recommendation system using LSTM and CNN. *Concurr. Comput. Pract. Exp.* **2022**, *34*, e7015. [CrossRef]
16. Koren, Y.; Bell, R.; Volinsky, C. Matrix Factorization Techniques for Recommender Systems. *Computer* **2009**, *42*, 30–37. [CrossRef]
17. Redhu, S.; Srivastava, S.; Bansal, B.; Gupta, G. Sentiment analysis using text mining: A review. *Int. J. Data Sci. Technol.* **2018**, *4*, 49–53. [CrossRef]
18. Pham, T.D.; Vo, D.; Li, F.; Baker, K.; Han, B.; Lindsay, L.; Pashna, M.; Rowley, R. Natural language processing for analysis of student online sentiment in a postgraduate program. *Pac. J. Technol. Enhanc. Learn.* **2020**, *2*, 15–30. [CrossRef]
19. Zheng, L.; Noroozi, V.; Yu, P.S. Joint Deep Modeling of Users and Items Using Reviews for Recommendation. In Proceedings of the WSDM'17: Tenth ACM International Conference on Web Search and Data Mining, Cambridge, UK, 6–10 February 2017; pp. 425–434. [CrossRef]
20. Fu, W.; Peng, Z.; Wang, S.; Xu, Y.; Li, J. Deeply Fusing Reviews and Contents for Cold Start Users in Cross-Domain Recommendation Systems. *Proc. AAAI Conf. Artif. Intell.* **2019**, *33*, 94–101. [CrossRef]
21. Xu, K.; Zhou, H.; Zheng, H.; Zhu, M.; Xin, Q. Intelligent classification and personalized recommendation of E-commerce products based on machine learning. *Appl. Comput. Eng.* **2024**, *64*, 147–153. [CrossRef]
22. Bellar, O.; Baina, A.; Ballafkih, M. Sentiment analysis: Predicting product reviews for E-commerce recommendations using deep learning and transformers. *Mathematics* **2024**, *12*, 2403. [CrossRef]
23. Xiang, A.; Huang, B.; Guo, X.; Yang, H.; Zheng, T. A Neural Matrix Decomposition Recommender System Model based on the Multimodal Large Language Model. *arXiv* **2024**, arXiv:2407.08942. [CrossRef]
24. Poriya, A.; Bhagat, T.; Patel, N.; Sharma, R. Non-Personalized Recommender Systems and User-based Collaborative Recommender Systems. *Int. J. Appl. Inf. Syst.* **2014**, *6*, 22–27. [CrossRef]
25. Xian, Y.; Yanzhong, D.; Jiangning, W. Analysis on the relationship between the number of product reviews and the number of purchases based on symbolic regression. *J. Syst. Eng.* **2020**, *35*, 289–300. (In Chinese) [CrossRef]
26. Lee, J.; Lee, J.N.; Shin, H. The long tail or the short tail: The category-specific impact of eWOM on sales distribution. *Decis. Support Syst.* **2011**, *51*, 466–479. [CrossRef]
27. Xia, H.; Wang, Y. A Product Recommendation Method by Analyzing Sales Volume, Sales Period, and User Satisfaction. In Proceedings of the 2024 12th International Conference on Information and Education Technology (ICIET), Yamaguchi, Japan, 18–20 March 2024; pp. 407–411. [CrossRef]
28. Rendle, S.; Freudenthaler, C.; Gantner, Z.; Schmidt-Thieme, L. BPR: Bayesian Personalized Ranking from Implicit Feedback. In Proceedings of the UAI'09: Twenty-Fifth Conference on Uncertainty in Artificial Intelligence, Montreal, QC, Canada, 18–21 June 2009.
29. Kingma, D.P.; Ba, J. Adam: A Method for Stochastic Optimization. *arXiv* **2014**, arXiv:1412.6980. [CrossRef]
30. Rakuten Group, Inc. Rakuten Ichiba Data. Informatics Research Data Repository, National Institute of Informatics. 2020. (Dataset). Available online: https://doi.org/10.32130/idr.2.1 (accessed on 21 May 2023).
31. He, X.; Deng, K.; Wang, X.; Li, Y.; Zhang, Y.; Wang, M. LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation. In Proceedings of the SIGIR'20: 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, Virtual Event, China, 25–30 July 2020; pp. 639–648. [CrossRef]
32. Kloberdanz, E.; Kloberdanz, K.G.; Le, W. DeepStability: A Study of Unstable Numerical Methods and Their Solutions in Deep Learning. In Proceedings of the ICSE'22: 44th International Conference on Software Engineering, Pittsburgh, PA, USA, 21–29 May 2022; pp. 586–597. [CrossRef]